

Kafka for a Java Tech Lead

Master Guide — Concepts • Java • Internals • Production • System Design • Interview

Audience: Senior Java Developers, Tech Leads and Architects

Objective: Move from “I have used Kafka” to confidently designing, troubleshooting, tuning and defending Kafka-based systems.

1. The Tech Lead Mental Model

Kafka is a distributed append-only log. Producers append records to partitions. Consumers read records by offset. Brokers persist and replicate partitions. Consumer groups distribute partitions among consumers.

- **Data model:** topic → partition → record → offset.
- **Scalability:** partitions are the main unit of parallelism.
- **Ordering:** guaranteed within a partition, not across a topic.
- **Durability:** replication, ISR and acknowledgement configuration.
- **Consumption:** groups, offsets, rebalancing and lag.
- **Lead responsibility:** semantics, failure handling, capacity, security and observability.

2. Core Architecture

Concept	Meaning	Interview point
Topic	Logical stream	Choose around domain/event semantics
Partition	Ordered log	Unit of parallelism and ordering
Offset	Position in partition	Defines progress and replay
Broker	Kafka server	Capacity and failure domains
Leader	Partition read/write leader	Leader distribution affects load
Follower	Partition replica	Provides redundancy
ISR	In-sync replicas	Important for safe durability
Consumer group	Shared work	One partition → one group member at a time

3. Producer — Java Usage

- **acks:** 0, 1, all. Critical events commonly use acks=all with suitable min.insync.replicas.
- **Idempotence:** enable.idempotence=true protects against duplicate producer writes caused by retries.
- **Retries:** handle transient broker/network failures.
- **Batching:** batch.size and linger.ms trade latency for throughput.
- **Compression:** reduces network/disk at CPU cost.
- **Keys:** normally determine partitioning and therefore per-key ordering.

```
Properties p = new Properties();
p.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "kafka1:9092,kafka2:9092");
p.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
p.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, JsonSerializer.class);
p.put(ProducerConfig.ACKS_CONFIG, "all");
p.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);

producer.send(new ProducerRecord<>("orders", orderId, event));
```

Interview trap: acks=all is not exactly-once. Acks controls acknowledgement/durability; idempotence protects producer retries. End-to-end semantics depend on the complete workflow.

4. Consumer Groups and Offsets

Consumers poll records. A consumer group lets multiple instances share partitions. Consequently, useful parallelism inside one group is bounded by the number of partitions.

- **max.poll.records:** controls batch size returned to application code.
- **max.poll.interval.ms:** important when processing can take a long time.
- **Heartbeats/session timeout:** influence failure detection.

- **Commit timing:** before processing risks loss; after processing can cause duplicates after crashes.
- **Rebalance:** partitions are reassigned when membership changes.

```
while (running) {
    ConsumerRecords<String, Event> records =
        consumer.poll(Duration.ofMillis(500));
    for (ConsumerRecord<String, Event> r : records) {
        processIdempotently(r.value());
    }
    consumer.commitSync();
}
```

5. Ordering, Keys and Partitioning

If all events for the same order, account or trade must be processed in order, use that business identifier as the record key. Kafka preserves order within the selected partition.

- More partitions increase parallelism but do not create global ordering.
- Hot keys can overload a single partition.
- Changing partition count can complicate ordering assumptions.
- Challenge any requirement for global ordering; often the real requirement is per entity.

6. Delivery Semantics

Semantic	Typical approach	Risk
At-most-once	Commit before processing	Possible loss
At-least-once	Process then commit	Duplicates possible
Exactly-once	Kafka transactions in a compatible flow	Scope and complexity

Lead insight: Kafka exactly-once does not make an arbitrary PostgreSQL update or external REST call exactly-once. For DB + Kafka consistency, discuss transactional outbox, idempotency and deduplication.

7. Replication and Durability

A partition has one leader and followers. Replication factor defines the number of copies. ISR contains replicas sufficiently caught up for the configured replication semantics.

- **RF=3**: common production starting point, subject to availability and cost requirements.
- **min.insync.replicas**: works with strong producer acknowledgements to protect durability when replicas fall out of sync.
- **Unclean leader election**: can trade data safety for availability; understand the consequence.
- Place replicas across independent failure domains where possible.

8. Retention vs Compaction

Retention controls how long/how much history remains. **Compaction** retains the latest value per key subject to compaction semantics.

- Retention suits event history and replay windows.
- Compaction suits latest-state/changelog topics.
- Compaction is asynchronous.
- Retention and compaction can be combined.

9. Kafka vs Queue vs Database

Need	Kafka strength	Important caveat
High-throughput streaming	Excellent	Partition/capacity design
Independent consumers	Excellent	Each group has its own offsets
Replay	Excellent	Storage cost
Request/reply	Possible	Often not the natural abstraction
DB transaction state	Not a replacement	Use DB + integration pattern
Per-key ordering	Excellent	No global ordering

10. Spring Kafka

Know **KafkaTemplate**, **ProducerFactory**, **ConsumerFactory**, **@KafkaListener**, **listener containers**, **serializers**, **error handlers**, **retries** and **DLT**.

```
@KafkaListener(topics = "orders", groupId = "order-service")
public void onMessage(OrderCreated event,
    Acknowledgment ack) {
    processIdempotently(event);
    ack.acknowledge();
}
```

For production, classify errors. Transient failures can be retried with bounded backoff; permanent validation/business errors should move to a DLT/error workflow instead of retrying forever.

11. Retry, DLT and Poison Messages

- **Transient**: temporary DB/network/throttling issue → retry with backoff.
- **Permanent**: invalid data or business rule → DLT/error workflow.
- **Poison message**: repeated failure that can block partition progress.
- DLT is not the end of the design: provide alerting, correction and replay.

12. Schema Evolution

Treat event schemas as APIs because consumers may read old records months later.

- Use explicit compatibility/versioning rules.
- Adding optional fields is generally safer than removing required fields.
- Do not expose internal DB entities as long-lived event contracts.
- Avro, Protobuf or JSON Schema plus a schema registry can enforce compatibility.
- Document identifiers, nullability, units, enums and semantic meaning.

13. Transactions and Exactly-Once

Kafka transactions can atomically publish multiple records and commit consumer offsets in a Kafka transaction. They are especially useful for consume-transform-produce pipelines.

```
producer.beginTransaction();
try {
    producer.send(new ProducerRecord<>("out", key, value));
    producer.sendOffsetsToTransaction(offsets, groupMetadata);
    producer.commitTransaction();
} catch (Exception e) {
    producer.abortTransaction();
}
```

Do not claim this makes arbitrary external systems exactly-once. For external DB/API effects, use idempotency or an appropriate integration pattern.

14. Performance and Capacity

- **Throughput:** events/sec and bytes/sec per partition/topic/broker.
- **Latency:** producer acknowledgement, broker request, consumer processing and end-to-end latency are different.
- **Partitions:** add parallelism but also metadata, recovery and operational overhead.
- **Batching/compression:** usually improve throughput; benchmark realistic payloads.
- **Consumer concurrency:** cannot usefully exceed available partitions within a group.
- **Storage:** retention + message size + replication drive disk requirements.
- **Network:** replication plus producer/consumer traffic drives broker network load.

Ingress bytes/sec = peak events/sec × average event bytes
Primary retained storage ≈ ingress bytes/sec × retention seconds
Replication storage ≈ retained primary storage × replication factor
Add headroom for bursts, recovery, compaction and operations.

15. Observability

Signal	Meaning	First checks
Consumer lag	Consumer is behind	Producer rate, processing time, downstream latency
Under-replicated partitions	Replication health issue	Broker/network/disk
Offline partitions	Availability issue	Broker/controller/replica state
Producer errors/latency	Publish-path problem	Broker saturation, network, retries
Disk usage	Capacity pressure	Retention, growth, replication
Rebalances	Group instability	Crashes, poll interval, deployments

16. Security

- **Authentication:** identify clients/brokers using platform-supported mechanisms such as SASL or mTLS.
- **Encryption:** TLS in transit; protect credentials and certificates.
- **Authorization:** ACLs/policies for topic, group and cluster operations.
- Use least privilege, secret rotation, environment isolation and careful handling of sensitive payloads.

17. Production Troubleshooting Scenarios

Lag suddenly increases: compare producer rate vs consumer throughput; inspect CPU/GC, downstream DB/API latency, rebalances and hot partitions. Scale only when partition and downstream capacity allow it. Then identify root cause.

Duplicate business transaction: inspect producer retries, consumer retries, commit timing and replay. Use a stable event/business ID and idempotent processing, often with a DB uniqueness constraint.

One record repeatedly fails: classify transient vs permanent; use bounded retry/backoff; send permanent failures to DLT; alert and provide replay after correction.

Broker disk fills: identify topic growth, retention, replication, oversized messages and producer spikes. Increase capacity or adjust retention only after confirming replay requirements.

18. System Design Pattern — DB + Kafka

```
Order API
|
v
Order Service --> PostgreSQL
|
| transactional outbox
v
Outbox Publisher --> Kafka: order-events
|
+-----+-----+
| |
v v
Payment Service Notification Service
|
v
Payment DB
```

Why outbox? The business DB change and outbox row are committed atomically. A publisher later sends the event. Consumers remain idempotent because delivery can be retried.

19. Interview Framework

- 1. Clarify throughput, ordering, durability, latency and replay.
- 2. Choose the business partition key.
- 3. Choose partition count and growth strategy.
- 4. Choose RF, acks and min.insync.replicas.
- 5. Select delivery semantics; often at-least-once + idempotent consumer.
- 6. Design retry, DLT, poison-message and downstream failure behavior.
- 7. Define lag, replication, latency and error monitoring.
- 8. Explain trade-offs and alternatives.

20. High-Value Interview Q&A;

Q: Why partitions?

A: They provide horizontal scalability, parallelism and the boundary for ordering.

Q: Can two consumers in one group read the same partition simultaneously?

A: No. A partition is assigned to one group member at a time.

Q: Can different groups read the same records?

A: Yes. Each group tracks its own offsets.

Q: Does Kafka guarantee ordering?

A: Only within a partition.

Q: How preserve order for an order ID?

A: Use the order ID as the message key.

Q: What happens when a consumer crashes?

A: Partitions are reassigned; uncommitted records may be processed again.

Q: Why monitor consumer lag?

A: It indicates how far processing trails production and exposes capacity/health problems.

Q: acks=all vs idempotence?

A: acks controls acknowledgement/durability; idempotence protects producer retries from duplicate writes.

Q: How handle duplicates?

A: Use idempotent business processing/deduplication; transactions where the workflow supports them.

Q: Kafka vs RabbitMQ/JMS?

A: Kafka emphasizes durable high-throughput logs, replay and fan-out; traditional queues often emphasize work distribution/routing.

Q: What is a DLT?

A: A separate topic/workflow for records that cannot be processed after bounded retries.

Q: What is a hot partition?

A: A partition receiving disproportionate traffic, commonly due to key skew.

Q: Why avoid excessive partition counts?

A: Partitions add metadata, recovery, file/network and operational overhead.

Q: What is ISR?

A: Replicas sufficiently caught up with the leader for configured replication semantics.

Q: Does RF=3 guarantee zero data loss?

A: No. It improves durability but depends on acknowledgement, ISR/min-ISR and failure behavior.

Q: When use Kafka Streams?

A: For stateful stream processing such as joins, aggregations, windows and Kafka-to-Kafka transformations.

21. Tech Lead Mastery Checklist

- Explain topic, partition, offset, broker, leader, ISR and consumer group.
- Design partition keys from business ordering requirements.
- Explain producer acks, retries and idempotence.
- Implement explicit offset strategy and idempotent consumers.
- Explain rebalancing and long-running processing risks.
- Design bounded retry + DLT workflows.
- Explain exactly-once and the external-system limitation.
- Explain RF, ISR, min.insync.replicas and leader election trade-offs.
- Explain retention vs compaction.
- Handle schema evolution as API governance.
- Perform high-level partition/storage/throughput sizing.
- Diagnose lag, replication and consumer failures.
- Explain authentication, TLS and authorization.
- Design Spring Kafka services and tests.
- Design DB → Kafka with an outbox.
- Lead a Kafka production incident and communicate trade-offs.

22. Recommended Learning Sequence

- **Phase 1:** topics, partitions, offsets, producers, consumers and groups.
- **Phase 2:** ordering, keys, commits, retries, idempotence and delivery semantics.
- **Phase 3:** replication, ISR, recovery, retention and compaction.
- **Phase 4:** Spring Kafka, serialization, error handling, DLT and testing.
- **Phase 5:** schema evolution, transactions, outbox and Kafka Streams.
- **Phase 6:** capacity, observability, security and incident troubleshooting.
- **Phase 7:** system design and mock interviews.

23. Final Mental Model

Requirement → Topic → Key → Partitions → Consumer Groups → Delivery Semantics → Failure Handling → Replication → Observability → Security → Trade-offs.

When an interviewer asks “How would you use Kafka?”, start with the business requirement, not configuration. Explain the partition key, delivery semantics, failure handling, replication, observability and trade-offs. That is the difference between having used Kafka and being able to own a Kafka architecture.

End of guide